

Contents

1	Zero to Hero Study Handbook: AI API Intelligence Agent	1
1.1	Module 1: Foundations & Architecture	1
1.1.1	1.1 What this project does	1
1.1.2	1.2 Core paradigms and patterns used here	1
1.1.3	1.3 Architecture and component interaction	2
1.2	Module 2: Repository Map	3
1.3	Module 3: Core Execution Flows	3
1.3.1	3.1 Flow A: FastAPI analysis request (POST /analyze)	3
1.3.2	3.2 Flow B: Planner, router, and connector execution	4
1.3.3	3.3 Flow C: Validation, reasoning, report generation, memory	4
1.3.4	3.4 Flow D: Reflection and completion status	5
1.3.5	3.5 Key input and output shapes (from real schemas)	5
1.4	Module 4: Setup & Run Guide	6
1.4.1	4.1 Prerequisites inferred from manifests	6
1.4.2	4.2 Environment configuration	7
1.4.3	4.3 Configuration files and what they control	7
1.4.4	4.4 Typical command sequences (documented project usage)	7
1.4.5	4.5 Database/migration/seeding notes	8
1.4.6	4.6 Deployment files	8
1.5	Module 5: Study Plan & Practice Exercises	8
1.5.1	5.1 Ordered study plan for new learners	8
1.5.2	5.2 Practice exercises	9
1.5.3	5.3 Solution outlines	9
1.6	Learner Verification Checklist	10

1 Zero to Hero Study Handbook: AI API Intelligence Agent

Project package name: `ai-api-intelligence-agent`.

1.1 Module 1: Foundations & Architecture

1.1.1 1.1 What this project does

This repository builds a production-style API intelligence agent that:

- Accepts a natural-language query.
- Selects one or more external APIs.
- Fetches and validates data asynchronously.
- Synthesizes analysis using a local Ollama model.
- Generates reports in multiple formats.
- Persists history and searchable memory.
- Exposes capabilities through FastAPI, Streamlit, and a Typer CLI.

Primary use cases from code paths:

- Interactive analysis via `POST /analyze` and `POST /query` in `src/api_intel_agent/api/app.py`.
- Direct provider access endpoints like `/github`, `/news`, `/weather`.
- Command-line workflows through `agent` commands in `src/api_intel_agent/cli/main.py`.
- Dashboard views in `streamlit_app/Home.py` and `streamlit_app/pages/*.py`.

1.1.2 1.2 Core paradigms and patterns used here

Definitions first:

- Object-Oriented Design: behavior grouped into classes with clear responsibilities.
- Pipeline / Workflow Orchestration: a fixed sequence of processing stages.
- Async I/O: non-blocking network calls using `async/await`.
- Adapter/Registry Pattern: lookup table maps names to connector/tool implementations.
- Schema-First Contracts: `pydantic` models define request/response structures.
- Graceful Degradation: system returns useful output even when one component fails.

How those appear in this repo:

- OOP: `AgentRuntime`, `ConnectorRegistry`, `AuthManager`, `MemoryManager`, `ReportGenerator`.
- Workflow orchestration: `IntelligenceGraph.compile()` in `src/api_intel_agent/agents/graph.py` defines node order.
- Async I/O: `BaseConnector.execute()` and `OllamaProvider.generate()` use `httpx.AsyncClient`.
- Registries:
 - Connectors: `ConnectorRegistry._connectors` in `src/api_intel_agent/connectors/registry.py`.
 - Tools: `ToolRegistry` and `build_tool_registry()` in `src/api_intel_agent/tools`.
- Schema contracts: `AnalyzeRequest`, `AnalyzeResponse`, `ConnectorResult` in `src/api_intel_agent/core/schemas.py`.
- Graceful fallback:
 - `AgentRunner.analyze()` falls back to deterministic runtime path if graph invocation fails.
 - `ReasoningAgent.run()` inserts fallback summary when LLM response is empty or errors.
 - `ChromaMemoryStore` disables itself on failure instead of crashing execution.

1.1.3 1.3 Architecture and component interaction

Major runtime components:

- Interfaces:
 - FastAPI app in `src/api_intel_agent/api/app.py`.
 - CLI app in `src/api_intel_agent/cli/main.py`.
 - Streamlit UI in `streamlit_app/`.
- Orchestration:
 - `AgentRunner` -> `IntelligenceGraph` -> node classes in `src/api_intel_agent/agents/`.
- Integrations:
 - API connectors in `src/api_intel_agent/connectors/`.
 - Local LLM client in `src/api_intel_agent/llm/ollama_provider.py`.
- State and persistence:
 - Memory: `SQLiteMemoryStore` + `ChromaMemoryStore` via `MemoryManager`.
 - Cache: `CacheManager` using SQLite or Redis backend.
- Analytics/reporting/observability:
 - `AnalyticsEngine`, `ReportGenerator`, `MetricsCollector`.

ASCII flow (main analysis path):

```

User (API/CLI/UI)
  |
  v
AnalyzeRequest (pydantic)
  |
  v
AgentRunner.analyze()
  |
  v
LangGraph: request_planner
          -> api_router
          -> authentication
          -> data_fetch
          -> validation

```

```

-> reasoning
-> report_generator
-> memory
-> reflection --retry--> data_fetch
    |
    +---end--> AnalyzeResponse
        |
        +--> artifacts/reports/*
        +--> artifacts/memory/*
        +--> artifacts/cache/*

```

1.2 Module 2: Repository Map

File/Directory Path	Primary Responsibility	Key Class
main.py	FastAPI process entrypoint	uvicorn
app.py	Streamlit process entrypoint	subproc
src/api_intel_agent/api/app.py	HTTP endpoints, auth guards, scheduler wiring	analyze
src/api_intel_agent/agents/runner.py	Orchestration entry for analysis	AgentRun
src/api_intel_agent/agents/graph.py	LangGraph node/edge definition	Intelli
src/api_intel_agent/agents/nodes.py	Node implementations for each agent step	Request
src/api_intel_agent/agents/state.py	Workflow state model	GraphSt
src/api_intel_agent/core/schemas.py	Shared data contracts	Analyze
src/api_intel_agent/connectors/base.py	Async HTTP connector base with retries/rate limit	BaseCon
src/api_intel_agent/connectors/providers.py	Provider-specific request mapping	GitHubC
src/api_intel_agent/connectors/registry.py	Connector lookup and query-to-provider inference	Connect
src/api_intel_agent/auth/manager.py	Connector auth headers + JWT/password functions	auth_he
src/api_intel_agent/api/security.py	FastAPI auth dependencies	get_cur
src/api_intel_agent/api/db.py	SQLAlchemy user table and DB init	User, in
src/api_intel_agent/cache/backends.py	Cache backend implementations	SQLiteC
src/api_intel_agent/memory/sqlite_store.py	Persistent query/response history storage	save_re
src/api_intel_agent/memory/chroma_store.py	Semantic memory search	upsert,
src/api_intel_agent/memory/manager.py	Unified memory API	store_a
src/api_intel_agent/analytics/engine.py	Rankings, distributions, chart generation	reposit
src/api_intel_agent/reporting/generator.py	Multi-format report generation	generat
src/api_intel_agent/monitoring/metrics.py	Runtime metrics snapshots	Metrics
src/api_intel_agent/llm/ollama_provider.py	Ollama REST wrapper	generat
src/api_intel_agent/tools/factory.py	Tool registration	build_t
src/api_intel_agent/tools/builtins.py	Built-in tools implementation	http_cl
src/api_intel_agent/tools/openapi_toolgen.py	Dynamic tool creation from OpenAPI	regist
src/api_intel_agent/cli/main.py	Typer CLI commands	query, g
streamlit_app/Home.py + streamlit_app/pages/*.py	Dashboard and analysis pages	page scri
configs/settings.yaml	Main runtime configuration	YAML s
.env.example	Environment variable template	Secret ke
Dockerfile + docker-compose.yml	Containerized deployment	API + S
k8s/*.yaml	Kubernetes deployment templates	Deploym
scripts/execute_notebook_sequential.py	Notebook execution helper	main
tests/*.py	Unit coverage for key modules	e.g., tes

1.3 Module 3: Core Execution Flows

1.3.1 3.1 Flow A: FastAPI analysis request (POST /analyze)

Step-by-step path:

1. Request enters `analyze(request: AnalyzeRequest, _user=Depends(get_current_user))` in `src/api_intel_agent/api/app.py`.
2. Endpoint calls `await runner.analyze(request)`.
3. `AgentRunner.analyze()` in `src/api_intel_agent/agents/runner.py`:
 - creates `GraphState(request=request)`.
 - invokes compiled `LangGraph` with `self.graph.ainvoke({"state": state.model_dump(mode="json")})`.
 - on exception, runs deterministic fallback via `self.runtime.run_once(state)` loop.
4. Graph executes nodes in `src/api_intel_agent/agents/graph.py` order:
 - `request_planner` -> `api_router` -> `authentication` -> `data_fetch` -> `validation` -> `reasoning` -> `report_generator` -> `memory` -> `reflection`.
5. `reflection` either ends or retries from `data_fetch` based on `_after_reflection()`.
6. Final state is converted by `GraphState.to_response()` and returned as JSON.

1.3.2 3.2 Flow B: Planner, router, and connector execution

Planner and router behavior in `src/api_intel_agent/agents/nodes.py`:

- `RequestPlannerAgent.run()`:
 - If `request.apis` is provided, filters valid providers from `ConnectorRegistry.list_names()`.
 - Else calls `ConnectorRegistry.infer_from_query(query)`.
 - Builds `state.plan_steps` as `PlanStep(provider, purpose, parallel_group=0)`.
- `APIRouterAgent.run()` creates fallback steps if plan is empty.

Fetch behavior in `DataFetchAgent.run()`:

- Groups `PlanStep` objects by `parallel_group`.
- Runs each group concurrently with `asyncio.gather`.
- Each connector call passes through `_run_connector()`:
 - cache lookup via `CacheManager.get("api_response", cache_key)`.
 - on miss, calls `await connector.execute(query, params)`.
 - writes cache on success/empty.
 - records latency with `MetricsCollector.record_api_latency`.

Provider execution contract in `BaseConnector.execute()` (`src/api_intel_agent/connectors/base.py`):

- Resolves auth via `AuthManager.auth_headers(provider)`.
- If auth status is `skipped_missing_credentials`, returns `ConnectorResult` with `status="skipped_missing_credentials"` and `ErrorRecord(code="MISSING_CREDENTIALS", retryable=False)`.
- Performs retrying HTTP call with tenacity in `_call_with_retry()`.
- Normalizes payload into `list[dict]`.
- Returns `ConnectorResult` with fields: `provider`, `endpoint`, `status`, `records`, `pagination`, `latency_ms`, `error`.

1.3.3 3.3 Flow C: Validation, reasoning, report generation, memory

Validation (`ValidationAgent.run()`):

- For each `ConnectorResult`:
 - If status is `failed` or `skipped_missing_credentials` and error exists, append to `state.errors`.
 - Else keep only dict records in `state.validated_records[provider]`.

Reasoning (`ReasoningAgent.run()`):

- Builds prompt from user query + `provider: record_count` summaries.
- Calls `await self.llm.generate(prompt, model=state.request.model)`.
- On error/empty response, uses fallback summary string.
- Generates deterministic recommendations list in code.

Report generation (`ReportGeneratorAgent.run()`):

- Uses AnalyticsEngine:
 - repository_rankings, language_distribution, api_latency_summary, generate_charts.
- Stores chart metadata in state.charts.
- Converts state.to_response() and generates every OutputFormat through ReportGenerator.generate().
- Writes report paths into state.report_paths keys: markdown, html, pdf, json, csv.

Memory persistence (MemoryAgent.run()):

- Calls MemoryManager.store_analysis(query, response):
 - SQLite: save query + response payload + provider summaries.
 - Chroma: upsert summary with metadata {query, status}.
- If request.use_memory is true, semantic search results are stored in telemetry under memory_hits.

1.3.4 3.4 Flow D: Reflection and completion status

ReflectionAgent.run():

- Adds completed_at timestamp to telemetry.
- Checks retryable errors.
- If retryable and below configured attempts (settings.agent.retry_max_attempts), increments state.retries and trims retryable errors.
- Else sets state.done = True.

GraphState.to_response() status mapping:

- RunStatus.SUCCESS: no errors.
- RunStatus.PARTIAL: both errors and connector results exist.
- RunStatus.FAILED: errors exist and no connector results.

1.3.5 3.5 Key input and output shapes (from real schemas)

1.3.5.1 AnalyzeRequest Defined in src/api_intel_agent/core/schemas.py:

```
{
  "query": "string",
  "model": "string|null",
  "apis": ["string"],
  "timeframe": "string|null",
  "output_format": "markdown|html|pdf|json|csv",
  "use_memory": true,
  "use_cache": true,
  "max_parallel_calls": 4
}
```

1.3.5.2 ConnectorResult

```
{
  "provider": "string",
  "endpoint": "string",
  "status": "ok|empty|failed|skipped_missing_credentials|...",
  "records": [{"any": "provider specific fields"}],
  "pagination": {"attempts": 1},
  "latency_ms": 12.34,
  "cached": false,
  "error": {
    "code": "string",
    "message": "string",
    "provider": "string|null",
  }
}
```

```

    "retryable": false
  }
}

```

1.3.5.3 AnalyzeResponse

```

{
  "run_id": "uuid-string",
  "status": "success|partial|failed|running",
  "summary": "string",
  "insights": ["string"],
  "recommendations": ["string"],
  "sources": [
    {
      "provider": "string",
      "endpoint": "string",
      "url": "string|null",
      "retrieved_at": "ISO datetime"
    }
  ],
  "charts": [
    {
      "title": "string",
      "kind": "bar|pie|line|area|heatmap|...",
      "path": "string"
    }
  ],
  "artifacts": {
    "markdown": "path",
    "html": "path",
    "pdf": "path",
    "json": "path",
    "csv": "path"
  },
  "telemetry": {"any": "metrics and metadata"},
  "errors": [
    {
      "code": "string",
      "message": "string",
      "provider": "string|null",
      "retryable": false
    }
  ]
}

```

1.4 Module 4: Setup & Run Guide

This section describes setup and startup commands documented by repository files. It does not execute code.

1.4.1 4.1 Prerequisites inferred from manifests

From `pyproject.toml`:

- Python: `>=3.12,<3.13`.

- Package manager/workflow: uv.
- Main runtime dependencies include FastAPI, LangGraph, LangChain/Ollama, HTTPX, SQLAlchemy, Streamlit, Typer, ChromaDB, Redis, Plotly, ReportLab, MLflow.
- CLI entrypoint: `agent = api_intel_agent.cli.main:app`.

1.4.2 4.2 Environment configuration

1. Copy env template:

```
cp .env.example .env
```

2. Required and optional keys from `.env.example` and `configs/settings.yaml`:

- Core auth secret:
 - `AGENT_JWT_SECRET`
- API tokens (optional/required per provider):
 - `GITHUB_TOKEN`, `HUGGINGFACE_TOKEN`, `NEWS_API_KEY`, `REDDIT_BEARER_TOKEN`, `NASA_API_KEY`
 - `JIRA_API_TOKEN`, `GITLAB_TOKEN`, `NOTION_TOKEN`, `GOOGLE_SHEETS_TOKEN`, `SLACK_BOT_TOKEN`, `GMAIL_TOKEN`
- Notifications:
 - `SLACK_WEBHOOK_URL`, `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`

3. Runtime override pattern (from `load_settings`):

- Any YAML key can be overridden by environment variable using prefix `AGENT__`.
- Example format: `AGENT__LLM__TIMEOUT_SECONDS=60`.

4. Feature toggle environment variables used in code:

- `AGENT_DISABLE_CHROMA=1` disables Chroma semantic memory (`src/api_intel_agent/memory/chroma_store.py`).
- `AGENT_DISABLE_SCHEDULER=1` disables periodic scheduler startup (`src/api_intel_agent/api/app.py`).

1.4.3 4.3 Configuration files and what they control

Main config: `configs/settings.yaml`.

Important sections:

- `llm`: base URL, default model, supported model aliases.
- `agent`: max iterations, parallel calls, retry/backoff.
- `auth`: JWT algorithm and token TTL.
- `cache`: backend (`sqlite` or `redis`), DB path, TTL.
- `memory`: SQLite path, Chroma path/enable flag, embedding model.
- `apis`: provider base URLs and env var names for auth.
- `monitoring`, `scheduler`, `ui`, `reports`.

1.4.4 4.4 Typical command sequences (documented project usage)

Install and environment setup:

```
uv python install 3.12
uv venv --python 3.12 .venv
source .venv/bin/activate
uv sync --all-groups --extra notebook
```

Start API service:

```
uv run python main.py
```

Start Streamlit dashboard:

```
uv run python app.py
```

Use CLI entrypoints:

```
uv run agent query "latest AI repositories"
uv run agent github
uv run agent weather --city London --lat 51.5 --lon -0.12
uv run agent news --topic AI
uv run agent search memory "llama"
```

Run notebook execution helper script:

```
uv run python scripts/execute_notebook_sequential.py \
  notebooks/zero_to_hero_api_intelligence_agent.ipynb \
  --output notebooks/zero_to_hero_api_intelligence_agent.executed.ipynb
```

1.4.5 4.5 Database/migration/seeding notes

- No separate migration framework is defined in repository files.
- User auth DB tables are auto-created by `init_db()` on FastAPI startup (`src/api_intel_agent/api/db.py`).
- SQLite memory and cache tables are auto-created by store constructors:
 - `SQLiteMemoryStore._init_db()`.
 - `SQLiteCache.__init__()`.
- Common artifact paths created automatically by settings loader and service classes:
 - `artifacts/auth/users.db`
 - `artifacts/memory/agent_memory.db`
 - `artifacts/memory/chroma/`
 - `artifacts/cache/cache.db`
 - `artifacts/reports/`
 - `artifacts/charts/`

1.4.6 4.6 Deployment files

- Dockerfile: Python 3.12 slim image, copies app and runs `uv run python main.py`.
- `docker-compose.yml`: two services (`api`, `streamlit`) exposing ports 8000 and 8501.
- k8s/ manifests:
 - `deployment.yaml`, `service.yaml`, `hpa.yaml`, `configmap.yaml`, `secret-template.yaml`.

1.5 Module 5: Study Plan & Practice Exercises

1.5.1 5.1 Ordered study plan for new learners

Recommended sequence:

1. Contracts first:
 - Read `src/api_intel_agent/core/schemas.py`.
 - Goal: understand all request/response and status models.
2. Runtime entrypoints:
 - Read `main.py`, `app.py`, `src/api_intel_agent/api/app.py`.
 - Goal: learn external interfaces and endpoint-to-core mapping.
3. Orchestration core:
 - Read `src/api_intel_agent/agents/state.py`, `graph.py`, `runner.py`, then `nodes.py`.
 - Goal: internal lifecycle from request planning to reflection.
4. Integration layer:
 - Read `src/api_intel_agent/connectors/base.py`, `providers.py`, `registry.py`.
 - Goal: how provider calls are built, retried, and normalized.
5. Cross-cutting systems:
 - Read `auth/manager.py`, `cache/backends.py`, `memory/*`, `reporting/generator.py`, `monitoring/metrics.py`.
 - Goal: reliability, persistence, and observability.

6. Interfaces and UX:

- Read CLI in `src/api_intel_agent/cli/main.py`.
- Read Streamlit pages in `streamlit_app/`.

7. Configuration and deployment:

- Read `configs/settings.yaml`, `.env.example`, `Dockerfile`, `docker-compose.yml`, `k8s/*.yaml`.

1.5.2 5.2 Practice exercises

1.5.2.1 Exercise 1 Question: In which exact order do LangGraph nodes run, and where is retry decision made?

Target files: `src/api_intel_agent/agents/graph.py`, `src/api_intel_agent/agents/nodes.py`.

1.5.2.2 Exercise 2 Question: If `NEWS_API_KEY` is missing, what status and error shape can a News connector response return?

Target files: `src/api_intel_agent/auth/manager.py`, `src/api_intel_agent/connectors/base.py`, `src/api_intel_agent/core/schemas.py`.

1.5.2.3 Exercise 3 Question: Trace `POST /query` end-to-end to the final `AnalyzeResponse` JSON.

Target files: `src/api_intel_agent/api/app.py`, `src/api_intel_agent/agents/runner.py`, `src/api_intel_agent/agent`

1.5.2.4 Exercise 4 Question: List all generated report formats and where each file path gets added to response artifacts.

Target files: `src/api_intel_agent/core/schemas.py`, `src/api_intel_agent/agents/nodes.py`, `src/api_intel_agent/reporting/generator.py`.

1.5.2.5 Exercise 5 Question: Explain cache key format and when cache hit metrics are updated.

Target files: `src/api_intel_agent/agents/nodes.py`, `src/api_intel_agent/cache/backends.py`, `src/api_intel_agent/monitoring/metrics.py`.

1.5.2.6 Exercise 6 Question: Add a new provider mentally (no coding): identify the minimum files and methods you would touch.

Target files: `src/api_intel_agent/connectors/providers.py`, `src/api_intel_agent/connectors/registry.py`, `configs/settings.yaml`.

1.5.2.7 Exercise 7 Question: Which endpoint requires `admin` role and how is role hierarchy enforced?

Target files: `src/api_intel_agent/api/app.py`, `src/api_intel_agent/api/security.py`, `src/api_intel_agent/api/db`

1.5.2.8 Exercise 8 Question: Identify all environment variables that can alter runtime behavior without code changes.

Target files: `.env.example`, `configs/settings.yaml`, `src/api_intel_agent/config/settings.py`, `src/api_intel_agent/api/app.py`, `src/api_intel_agent/memory/chroma_store.py`.

1.5.3 5.3 Solution outlines

Exercise 1 outline:

- Order is set in `IntelligenceGraph.compile()`.
- Retry is controlled by conditional edge on `reflection` through `_after_reflection()`.

Exercise 2 outline:

- `AuthManager.auth_headers("news")` returns `skipped_missing_credentials` if token missing.
- `BaseConnector.execute()` returns `ConnectorResult` with `status="skipped_missing_credentials"` and error code `MISSING_CREDENTIALS`.

Exercise 3 outline:

- `/query` builds `AnalyzeRequest` then calls `runner.analyze`.
- Runner invokes graph (or fallback runtime).
- Final conversion occurs in `GraphState.to_response()`.

Exercise 4 outline:

- `OutputFormat` enum defines `MARKDOWN`, `HTML`, `PDF`, `JSON`, `CSV`.
- `ReportGeneratorAgent.run()` loops for `fmt` in `OutputFormat` and stores paths in `state.report_paths[fmt.value]`.

Exercise 5 outline:

- Cache key: `f"{step.provider}:{query}:{step.params}"`.
- `record_cache(True/False)` called in `_run_connector` after cache lookup.

Exercise 6 outline:

- Implement `NewProviderConnector(BaseConnector)` with `build_request` (and optional `normalize`).
- Register in `ConnectorRegistry._connectors`.
- Add `apis.new_provider` config block in `configs/settings.yaml` with `base_url` and `auth_env`.

Exercise 7 outline:

- `/tools/openapi` is protected by `Depends(require_role("admin"))`.
- `require_role` compares numeric hierarchy mapping `{viewer:1, analyst:2, admin:3}`.

Exercise 8 outline:

- Secrets/tokens from `.env.example`.
- Hierarchical runtime overrides via `AGENT_...` from `_apply_env_overrides`.
- Behavior toggles include `AGENT_DISABLE_SCHEDULER` and `AGENT_DISABLE_CHROMA`.

1.6 Learner Verification Checklist

Use this checklist after studying:

- Can you explain `AnalyzeRequest` and `AnalyzeResponse` fields without opening the file?
- Can you describe the exact `LangGraph` node order and retry branch?
- Can you trace one request from FastAPI endpoint to persisted memory records?
- Can you explain how `BaseConnector.execute()` handles auth, retries, normalization, and errors?
- Can you list where cache, memory, report, and chart artifacts are written?
- Can you explain how JWT auth and role checks are enforced in API dependencies?
- Can you identify how to add a new API provider with minimal changes?
- Can you map each CLI command to the module/function it calls?
- Can you explain how environment overrides (`AGENT_...`) are applied to config?
- Can you describe at least one graceful-degradation path in this codebase?